



KOSTÜ
KOCAELİ SAĞLIK
VE TEKNOLOJİ
ÜNİVERSİTESİ
2009



**MÜHENDİSLİK VE DOĞA
BİLİMLERİ FAKÜLTESİ**

Proje Yönetimi-9

Dr. Öğr. Üyesi Ercan ÖLÇER

Kalite Güvence

- Giriş
- Test
 - o Test Aşamaları ve Yaklaşımları
 - o Kara Kutu Testi
 - o Beyaz Kutu Testi
 - o Sistem Testi

Kalite Güvencesi (QA) Nedir?

QA, önceden tanımlanmış kalite standartlarının yerine getirilmesini sağlamak için planlı ve plansız faaliyetlerin birleşimidir.

- QA'ya yapıcı ve analitik yaklaşımlar
 - Niteliksel ve niceliksel kalite standartları
 - Ölçüm
 - Ölçülebilir nicel faktörlerden nitel faktörlerin türetilmesi
- Yazılım Metrikleri

Kalite Güvence Yaklaşımları

- Yapıcı Yaklaşımlar

Bazı kalite faktörlerinin yerine getirilmesini sağlayan yöntemlerin, dillerin ve araçların kullanılması.

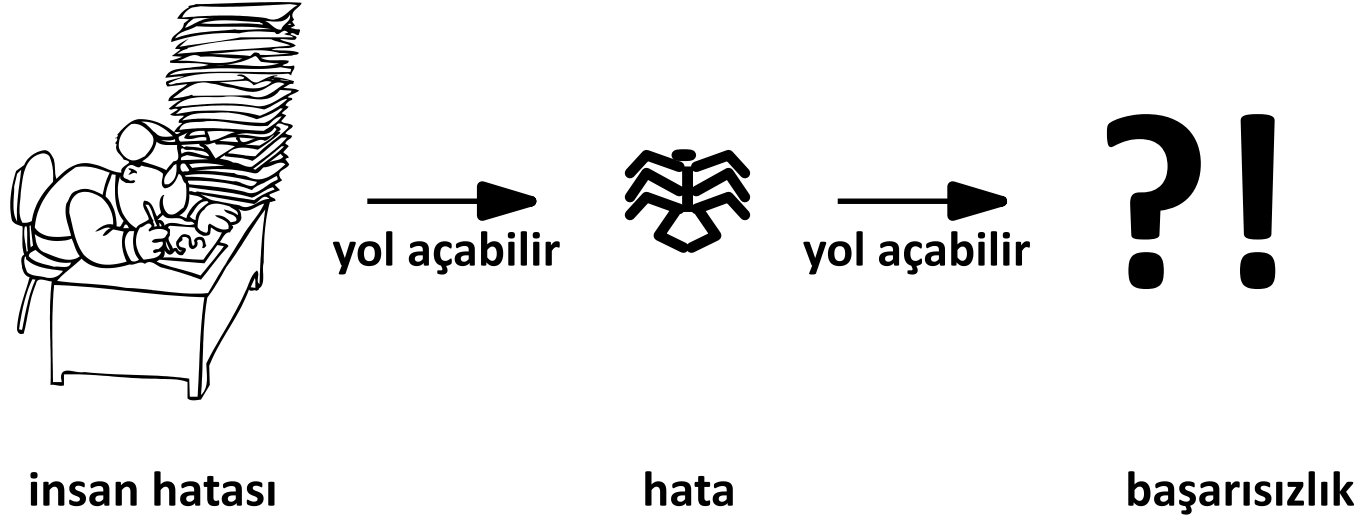
- Sözdizimine yönelik düzenleyiciler
- Tip sistemleri
- Dönüşümsel programlama
- Kodlama yönergeleri
- ...

- Analitik yaklaşımlar

Mevcut kalite seviyesini gözlemlemek için yöntemlerin, dillerin ve araçların kullanımı.

- Statik analiz araçları (örn. *lint*)
- Test
- ...

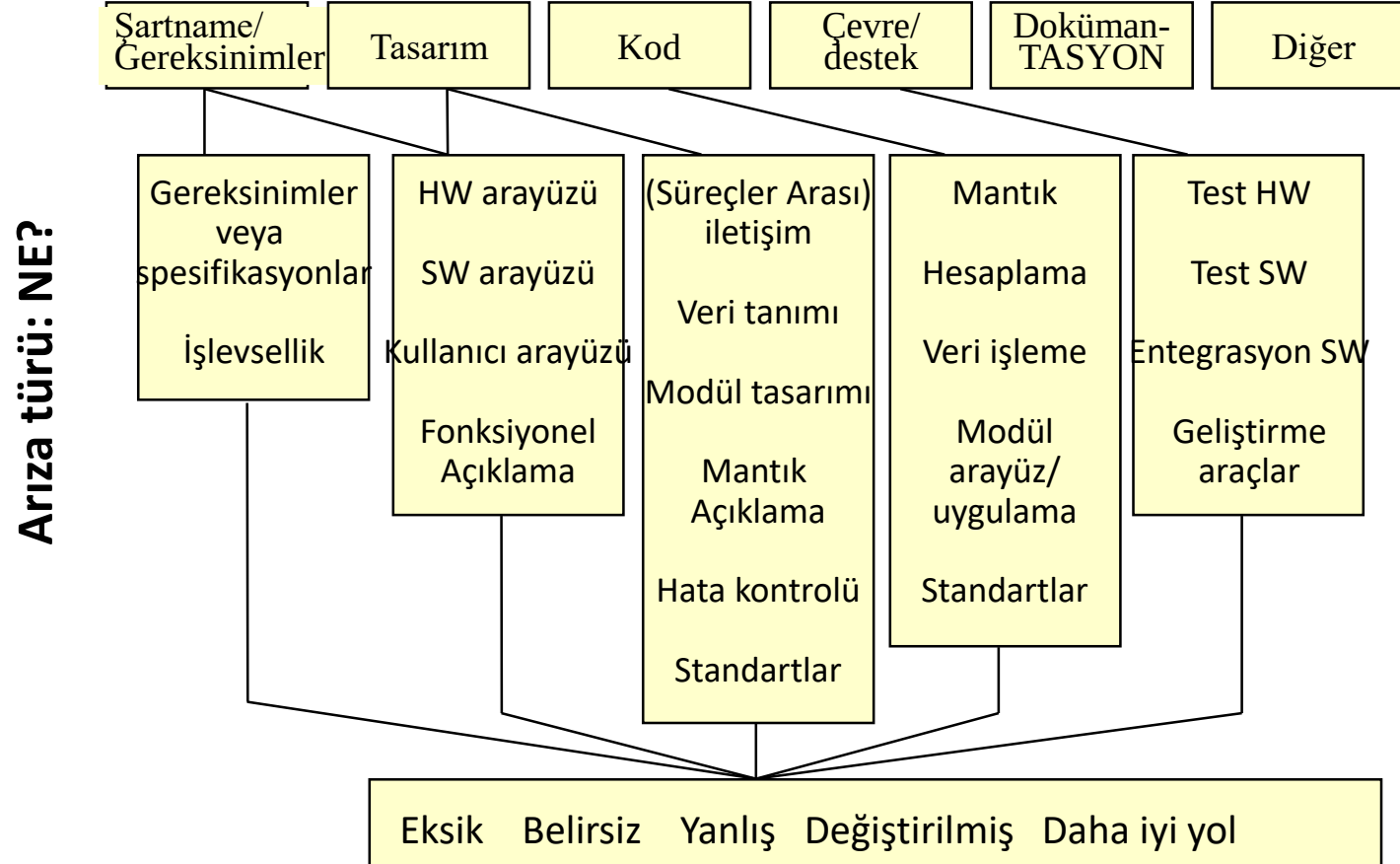
Hata vs Arıza



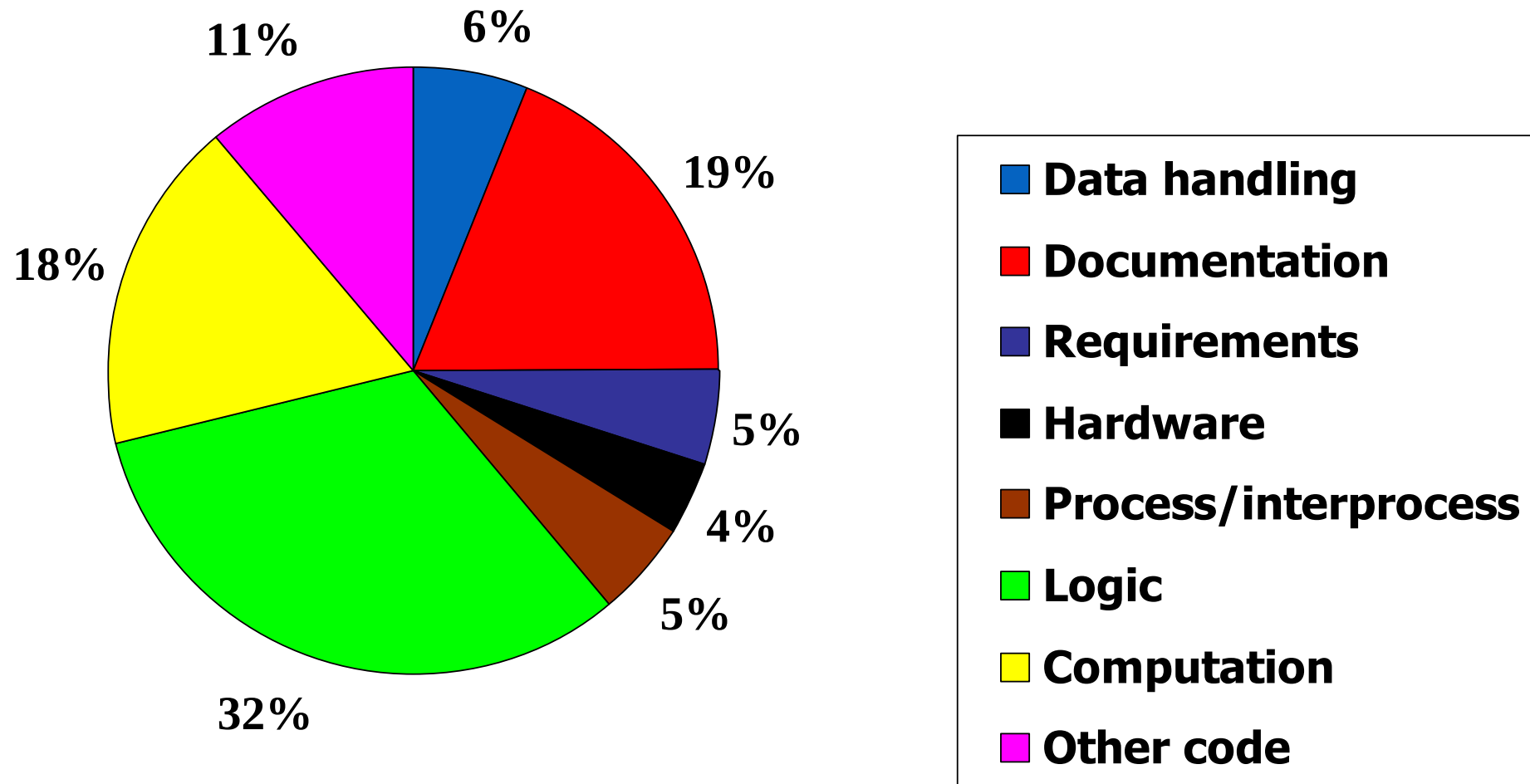
- Farklı arıza türleri
 - Farklı tanımlama teknikleri
 - Farklı test teknikleri
- Arıza önleme ve tespit stratejileri beklenen arıza profiline dayanmalıdır

Arıza Sınıflandırması

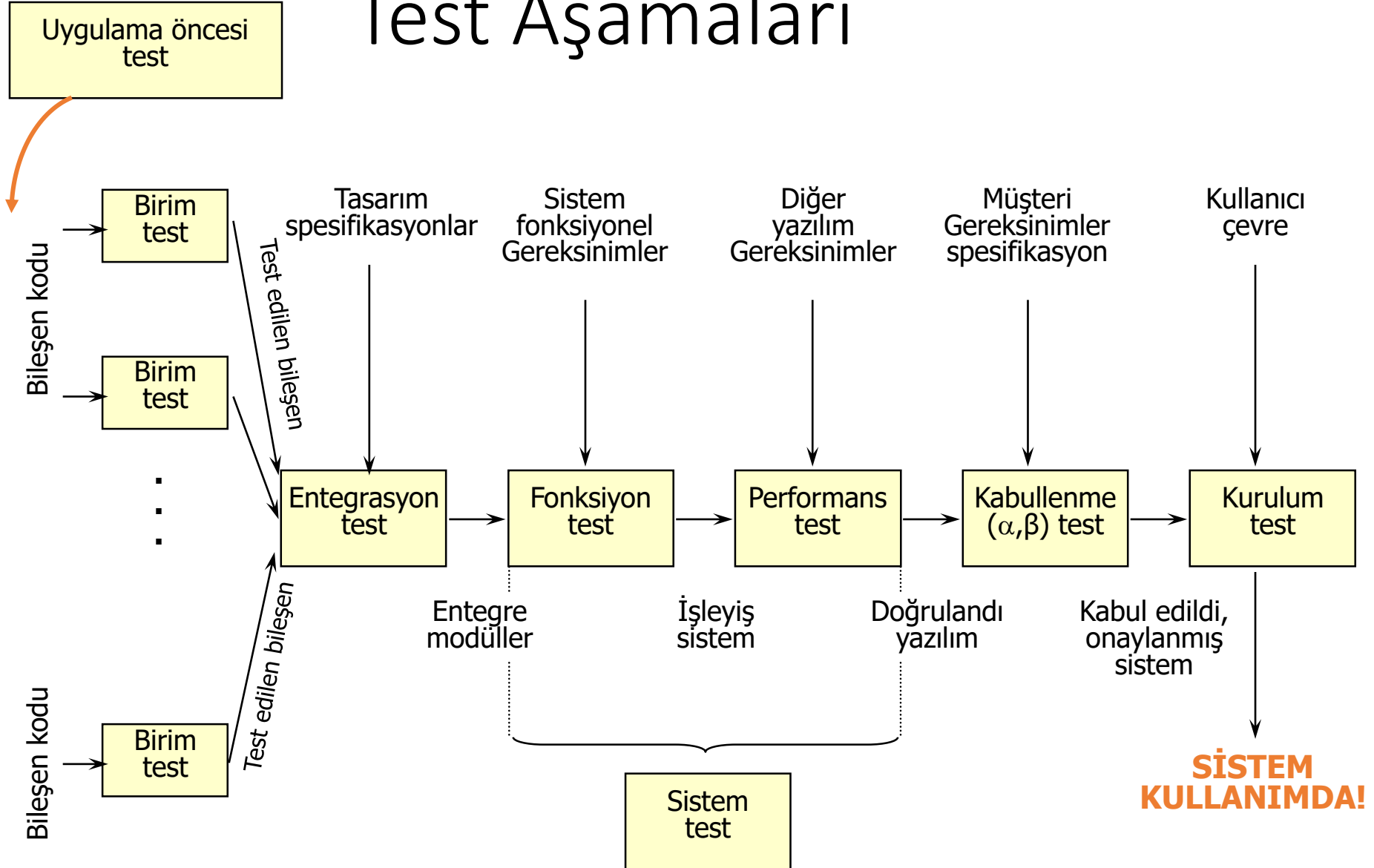
Hatanın kaynağı: NEREDE?



Bir Arıza Profili



Test Aşamaları



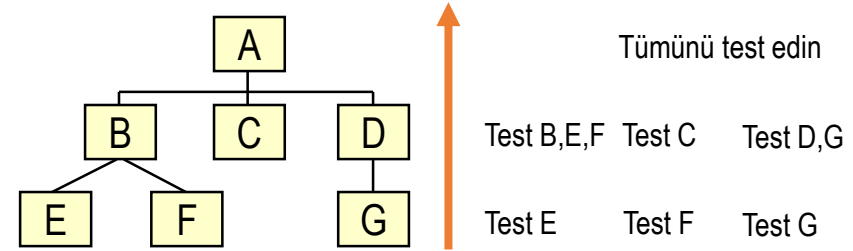
Uygulama Öncesi Testler

- Denetimler
 - Teknik inceleme veya test öncesinde belgelerin ve kodun değerlendirilmesi
- İzlenecek yol
 - Ekipler halinde
 - Kaynak kodu/detaylı tasarımı inceleyin
- Gözden Geçirmeler
 - Daha gayri resmi
 - Genellikle belge sahipleri tarafından yapılır
- Avantajlar
 - Yüksek öğrenme etkisi
 - Sistem bilgisinin dağıtılması

Entegrasyon Testi

Sistemin nasıl entegre edileceği ve test edileceği

- Yukarıdan aşağıya
- Aşağıdan yukarıya
- Önce kritik birimler
- İşlevsellik odaklı (iş parçacıkları)



Yapı düzeninin etkileri

- Yukarıdan aşağıya => taslaklar; daha kapsamlı üst düzey
- Aşağıdan yukarıya => sürücüler; daha kapsamlı alt düzey
- Kritik => saplamalar ve sürücüler.

Testin diğer aşamaları

- Sistem testi: Tüm sistemin işlevselliğini, performansını, güvenilirliğini ve güvenliğini test edin.
- Kabul testi: Sistemi kullanıcı ortamında, standart kullanıcı girdi senaryoları ile çalıştırın.
- Regresyon testi: Önceden onaylanmış sistemin değiştirilmiş versiyonlarının test edilmesi.

Test Etme ve Doğruluğu "Kanıtlama"

- Doğrulama (Verification)
 - Tasarımı/kodu gereksinimlere göre kontrol edin
 - Ürünü doğru inşa ediyor muyuz?
- Geçerleme (Validation)
 - Ürünü müşterinin beklentilerine göre kontrol edin
 - Doğru ürünü inşa ediyor muyuz?
- Test

Test, hataları bulmak amacıyla (muhtemelen tamamlanmamış) bir programın yürütüldüğü süreçtir.

[Myers 76]

Testler sadece hataların varlığını gösterebilir, yokluğunu asla gösteremez.

[Dijkstra 69]

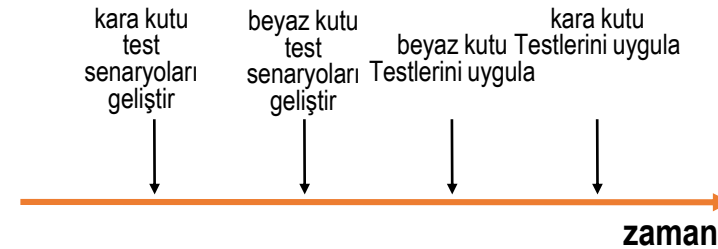
- Testler ne bir programın hatasız olduğunu ne de doğru olduğunu kanıtlayabilir

Test Prensipleri

- Test takımlarının oluşturulması
 - Hataları bulmak için varsayım altında testler planlayın
 - Tipik ve tipik olmayan girdileri deneyin
 - Girdi sınıfları oluşturun ve her sınıfın temsilcilerini seçin
- Testlerin gerçekleştirilmesi
 - Test uzmanları $\neq$ uygulayıcılar
 - Bir test çalıştırmadan önce beklenen sonuçları tanımlayın
 - Gereksiz hesaplama olup olmadığını kontrol edin
 - Test sonuçlarını iyice kontrol edin
 - Testi ayrıntılı olarak belgeleyin
- Testi basitleştirin
 - Programları ayrı ayrı test edilebilir birimlere bölme
 - Test dostu programlar geliştirin
- Her test tekrarlanabilir olmalıdır!

Test Yöntemleri

- Yapısal testler (white-box, glass-box)
 - Test senaryoları geliştirmek için kod/detaylı tasarım kullanır
 - Genellikle birim testlerinde kullanılır
 - Yaklaşımlar:
 - Kapsama dayalı test
 - Sembolik yürütme
 - Veri akışı analizi
 - ...
- İşlevsel test (black-box)
 - Test senaryoları geliştirmek için fonksiyon spesifikasyonlarını kullanır
 - Genellikle sistem testlerinde kullanılır
 - Yaklaşımlar:
 - Eşdeğerlik bölümlemesi
 - Sınır vaka analizi
 - ...



Test Hazırlığı

- Test senaryo kombinasyonlarının çokluğu nedeniyle kapsamlı testler yapılmaz.

➤Temsili test verileri seçin.

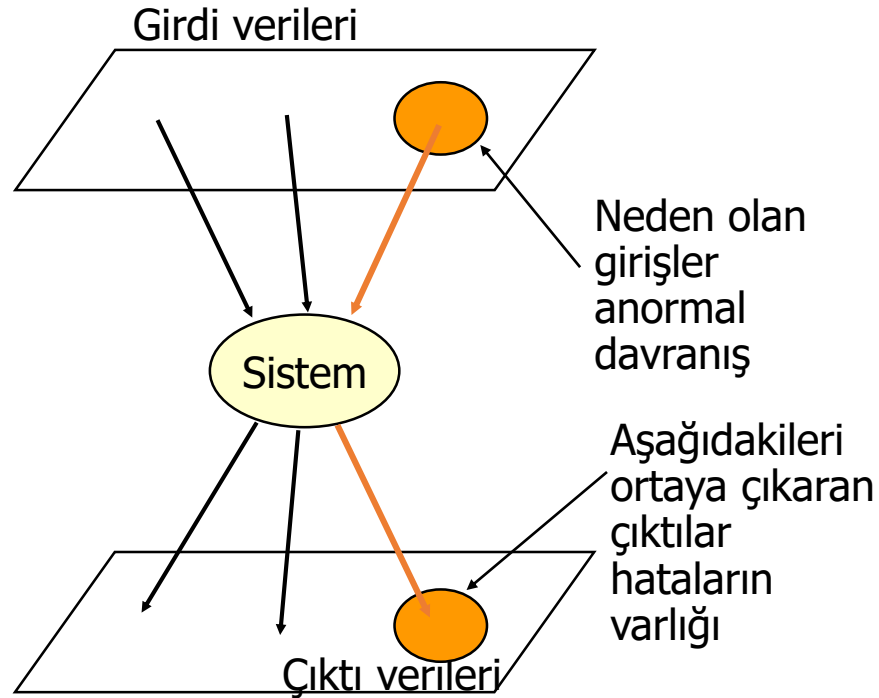
for i := 1 to 100 do if a = b then X else Y;	test etme adımları	#test
	1 X, Y	2
	2 XX, XY, YX, YY	4
	3 XXX, XXY, ...	8
	...	
	100	2^{100}

➤Test verilerini seçin (test *senaryoları*)

Test Kutusu Geliştirme

- Problemler:
 - Test senaryoları geliştirmenin sistematik yolu?
 - Tatmin edici bir test senaryosu kümesi?
 - Test senaryosu $\neq$ test verileri
 - Test verileri: Sistemi test etmek için tasarlanan girdiler
 - Test durumu:
 - Test edilecek durum
 - Bu durumu test etmek için girdiler
 - Beklenen çıktılar
- ➔ Test tekrarlanabilir
- ➔ Eşdeğerlik bölümlemesi
- ➔ Kapsama dayalı test

Eşdeğerlik Bölümleme



Girdi ve çıktı verileri, sınıftaki tüm üyelerin karşılaştırılabilir bir şekilde davrandığı sınıflar halinde gruplandırılabilir.

- Sınıfları tanımlayın
- Temsilcileri seçin
 - ◆ Tipik eleman
 - ◆ Sınırdaki vakalar

$x \in [25 .. 100]$

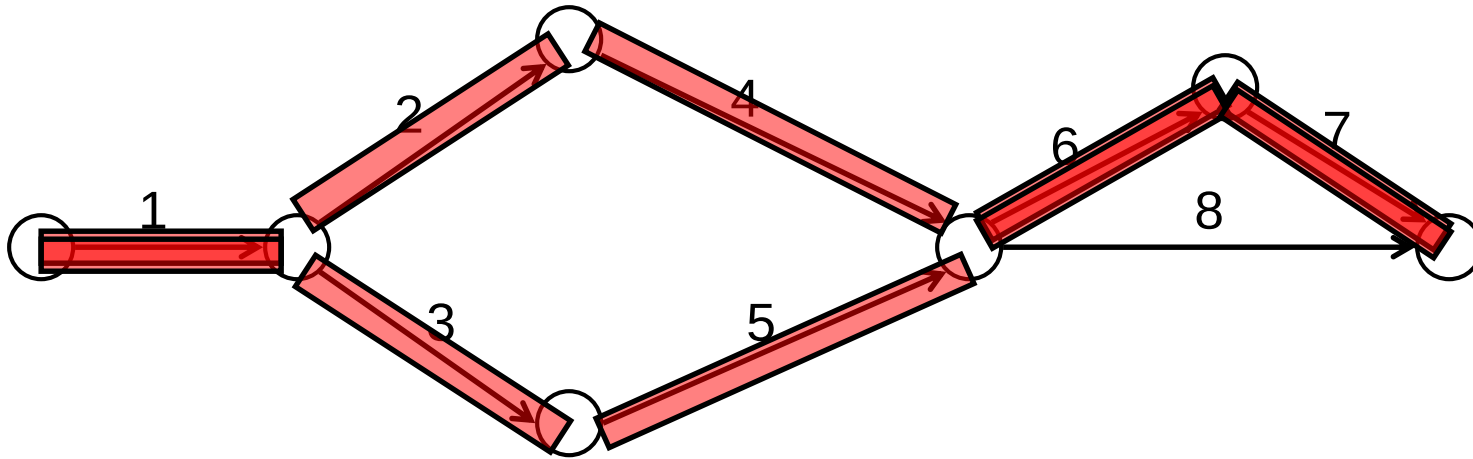
- sınıf 1: $x < 25$
- sınıf 2: $x \geq 25$ ve $x \leq 100$
- sınıf 3: $x > 100$

Kapsama Dayalı Test

- Avantajlar
 - Test senaryoları geliştirmenin sistematik yolu
 - Ölçülebilir sonuçlar (kapsam)
 - Kapsamlı araç desteği
 - Akış grafiği oluşturucular
 - Test verisi üreticileri
 - Defter Tutma
 - Dokümantasyon desteği
- Dezavantajlar
 - Kod mevcut olmalıdır
 - Veri odaklı programlar için (henüz) iyi çalışmıyor

Kapsama Dayalı Test - İfade Kapsamı

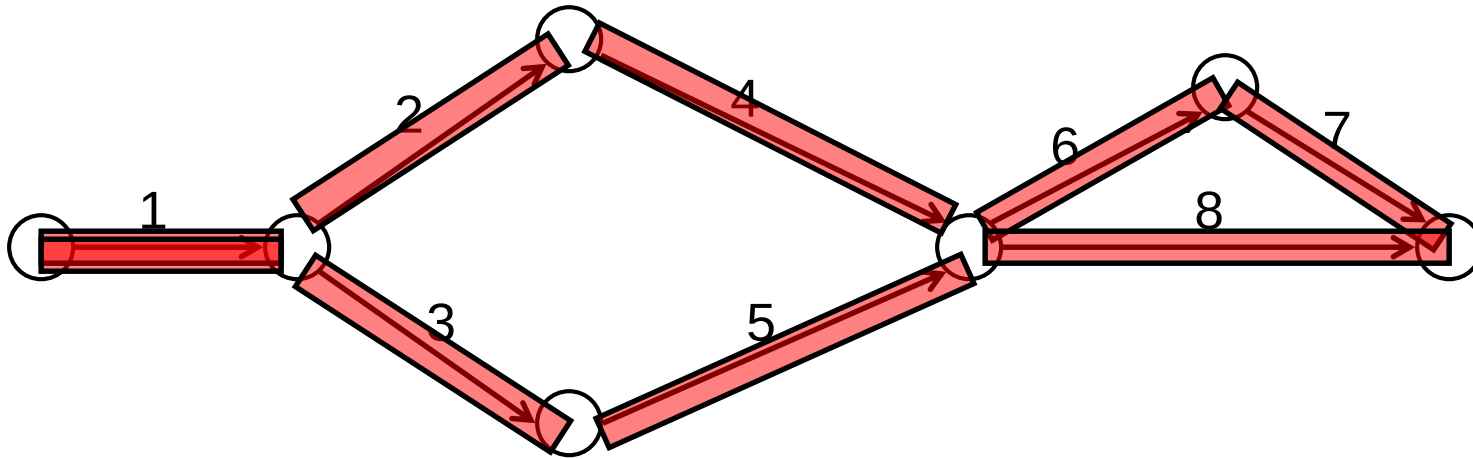
Her ifade bazı testlerde en az bir kez yürütülür



2 test vakası: 12467; 13567

Kapsama Dayalı Test - Dal Kapsamı

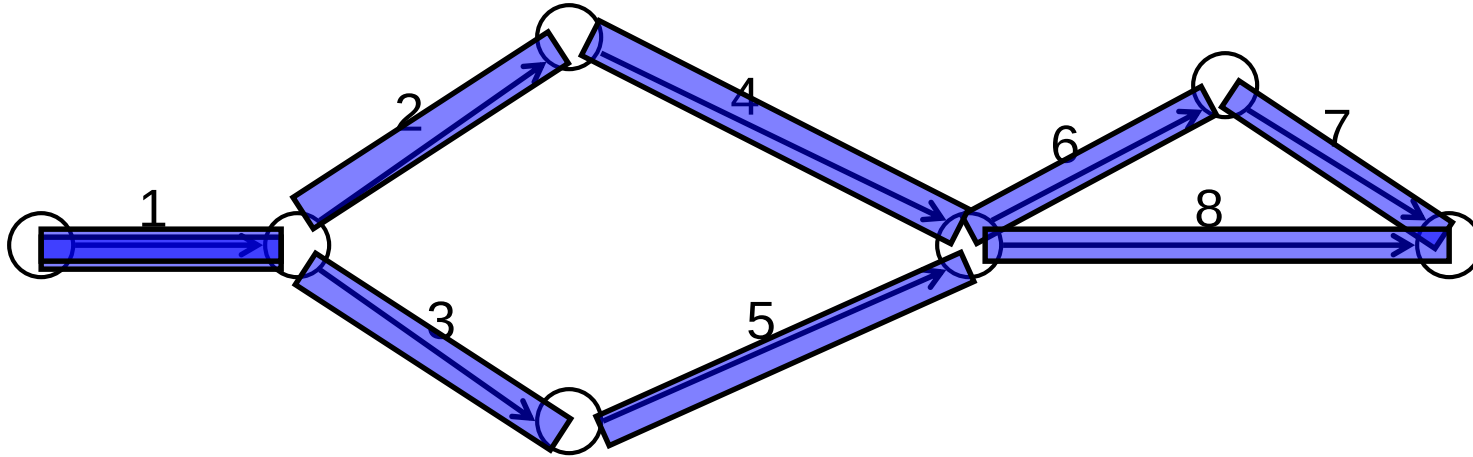
Grafikteki her karar noktası için, her dal en az bir kez seçilir



2 test vakası: 12467; 1358

Kapsama Dayalı Test - Yol Kapsamı

Kontrol akışı grafiğindeki her farklı yolun bazı testlerde en az bir kez yürütüldüğünden emin olun.



4 test vakası: **12467**; **1358**; **1248**; **13567**

Sınıf Çalışması 1: Eşdeğerlik Bölümlemesi

Problem:

Bir online alışveriş sitesinde 'indirim kodu' alanı için test senaryoları geliştirmeniz isteniyor. İndirim kodları aşağıdaki kurallara göre çalışır:

- İndirim kodu 6-10 karakter arası olmalı
- Sadece harf ve rakam içermeli (özel karakter yok)
- En az 2 harf içermeli
- Geçerli kodlar: %10, %20, %50 indirim sağlar

Çalışma 1: Görev

Adım 1: Eşdeğerlik Sınıfları

- Geçerli ve geçersiz sınıfları belirleyin:
 1. Karakter uzunluğu
 2. Karakter tipi
 3. Harf sayısı
 4. Kod geçerliliği

Adım 2: Test Senaryoları

- Her sınıf için temsili test verileri seçin:
 1. Tipik değerler
 2. Sınır değerler
 3. Geçersiz değerler

Çalışma 1: Örnek Çözüm

Özellik	Geçerli Sınıflar	Geçersiz Sınıflar	Test Örnekleri
Uzunluk	6-10 karakter	< 6, > 10	ABC123 (6), SALE2024 (8), WXYZ123456 (10), AB (2), VERYLONGCODE (12)
Karakter	Harf + Rakam	Özel karakter	ABC123 (✓), SALE20 (✓), ABC@123 (X)
Harf Sayısı	≥ 2 harf	< 2 harf	AB1234 (✓), A12345 (X), 123456 (X)
Geçerlilik	Sistemde kayıtlı	Kayıtsız kod	SUMMER10 (✓), FAKE123 (X)

Çalışma 2: Test Senaryo Tasarımı

Senaryo: Kullanıcı Giriş Modülü

1. Kullanıcı adı: 5-20 karakter, sadece harf, rakam ve alt çizgi (_)
2. Şifre: En az 8 karakter, en az 1 büyük harf, 1 küçük harf, 1 rakam
3. 3 başarısız denemeden sonra hesap 15 dakika kilitlenir
4. Boş alan kontrolü: Her iki alan da dolu olmalı
5. 'Beni Hatırla' seçeneği işaretlenirse oturum 30 gün açık kalır

Görev: Bu modül için 10 farklı test senaryosu tasarlayın (pozitif + negatif)

Test Senaryo Şablonu

Test ID	Test Adı	Ön Koşul	Test Adımları	Beklenen Sonuç
TC-001	Geçerli giriş	Kayıtlı kullanıcı	1. Kullanıcı adı gir 2. Şifre gir 3. Giriş tıkla	Ana sayfaya yönlendir
TC-002	Hatalı şifre	Kayıtlı kullanıcı	1. Doğru kullanıcı adı 2. Yanlış şifre 3. Giriş tıkla	Hata mesajı göster
...	...	...	...	...

Çalışma 3: Kapsama Analizi

Verilen Kod:

```
function hesaplaIndirim(tutar, musteriTipi, kuponVar) {  
    let indirim = 0;  
  
    if (tutar > 1000) {  
        if (musteriTipi === "VIP") {  
            indirim = tutar * 0.20;  
        } else {  
            indirim = tutar * 0.10;  
        }  
    }  
  
    if (kuponVar) {  
        indirim = indirim + 50;  
    }  
  
    return tutar - indirim;  
}
```

Çalışma 3: Görev

1. İfade Kapsamı

Tüm satırları en az bir kez çalıştıran test verilerini belirleyin

2. Dal Kapsamı

Tüm if/else dallarını kapsayan test verilerini tasarlayın

3. Yol Kapsamı

Tüm olası yolları kapsayan test verilerini oluşturun

Çalışma 3: Örnek Çözüm

Kapsama Tipi	Test Verisi	Kapsanan Satırlar/Dallar	Kapsama %
İfade Kapsamı	tutar=1500, musteriTipi='VIP', kuponVar=true	Tüm satırlar çalışır (1-16)	100%
		tutar=500, musteriTipi='Normal', kuponVar=false	Alternatif yol (sadece son return)
Dal Kapsamı	Set 1: tutar=1500, musteriTipi='VIP', kuponVar=true	True dalları: tutar>1000, VIP, kuponVar	100%
		Set 2: tutar=500, musteriTipi='Normal', kuponVar=false	False dalları: tüm koşullar
Yol Kapsamı	4 farklı test seti gerekli	Yol 1-4: Tüm kombinasyonlar	100%

Çalışma 4: Black-box vs White-box Test

Senaryo: Bir e-ticaret uygulamasının 'sepete ekle' özelliği

Black-box Test Yaklaşımı

Kod yapısına bakmadan, sadece işlevsel gereksinimlere göre test edin:

- Normal ürün ekleme
- Stok dışı ürün
- Maksimum miktar aşımı
- Negatif miktar girişi
- Misafir kullanıcı

White-box Test Yaklaşımı

Kod yapısını inceleyerek, tüm kod yollarını test edin:

- Tüm if-else dalları
- Döngü sınır değerleri
- Exception handling
- Veritabanı bağlantı kontrolü
- Bellek yönetimi

Çalışma 5: Gerçek Proje Test Planı

Proje: Mobil Bankacılık Uygulaması

- Modüller: Giriş, Hesap Görüntüleme, Para Transferi, Fatura Ödeme, QR Ödeme
- Test edilecek platformlar: iOS (14+), Android (10+)
- Güvenlik: 2FA, Parmak izi, Yüz tanıma
- Performans: Sayfa yüklenme < 2 saniye
- Kullanılabilirlik: WCAG 2.1 AA uyumluluğu

Test Planı Oluşturun

1

Test Kapsamı Belirleme

Hangi modüller test edilecek? • Hangi test türleri uygulanacak? • Test dışı kalacak alanlar?

2

Test Stratejisi

Black-box: Hangi senaryolar? • White-box: Hangi kod alanları? • Manuel vs Otomatik test oranı?

3

Test Takvimi

Birim testler: 1 hafta • Entegrasyon testleri: 1 hafta • Sistem testleri: 2 hafta • UAT: 1 hafta

4

Kaynaklar ve Roller

Test lideri: 1 kişi • Test uzmanı: 3 kişi • Test araçları ve ortamı

Bonus: Test Metrikleri ve KPI'lar

Metrik	Formül	Hedef
Test Kapsamı	$(\text{Test edilen özellik} / \text{Toplam özellik}) \times 100$	$\geq 90\%$
Hata Tespit Oranı	$(\text{Testte bulunan hata} / \text{Toplam hata}) \times 100$	$\geq 85\%$
Test Verimliliği	$(\text{Geçen test} / \text{Toplam test}) \times 100$	$\geq 95\%$
Hata Yoğunluğu	Bulunan hata / Kod satırı (KLOC)	$\leq 2/\text{KLOC}$

Bu metrikleri kendi projelerinizde nasıl kullanırsınız?